UCLUSTERs

CIUK 2026 · EPCC Cluster Challenge

Balancing performance and energy efficiency on Cirrus

Cirrus Specifications

CPUs	2 x AMD EPYC 9825
Physical cores	288 per node
Base frequency	2.2 GHz
Standard-node memory	768 GB nominal capacity
NUMA regions	8 per node, with 36 cores each
CCDs	24 per node, with 12 cores and 32 MB shared L3 cache each
Network	HPE Slingshot 11; two 200Gb/s injection ports per node

Established environment

HPL	Netlib HPL 2.3
HPCG	Reference HPCG 3.1
Compiler	Cray CCE 19.0.0
MPI	Cray MPICH 8.1.32
HPL mathematical library	Cray LibSci 25.03.0 (threaded)
Initial compiler optimisation	-O3

Benchmark Specifications

HPL (Dense linear algebra, compute-bound)

- Solves a dense system \(Ax=b\) using LU factorisation.
- Reuses matrix blocks repeatedly from cache.
- Performs many floating-point operations per byte moved.
- Therefore benefits directly from Cirrus's 288 cores and AVX-512 units.

HPCG (Sparse solver, memory-bound)

- Solves a sparse 3D system using multigrid-preconditioned conjugate gradient.
- Each grid point interacts with its 26 neighbours.
- Performs relatively little arithmetic before loading more values and indices.
- Limited mainly by DDR5 bandwidth, cache/NUMA locality and communication, rather than the CPU's arithmetic ceiling.

Expected HPCG performance

With approximately 1,228 GB/s of theoretical memory bandwidth and an estimated arithmetic intensity of 0.13 FLOP/byte, a Cirrus node has a simplified HPCG performance ceiling of around 160 GFLOP/s.

This helps explain why HPCG achieves much lower GFLOP/s than compute-bound HPL.

HPL Benchmark Experiment - Optimising for Performance & Power Efficiency

Methodology

We tuned HPL by changing one parameter at a time. In each experiment, every setting was fixed except a single independent variable, such as the problem size N, the block size NB, the MPI rank × OpenMP thread layout, or the number of nodes. We ran that variable at five different values, one value per job.

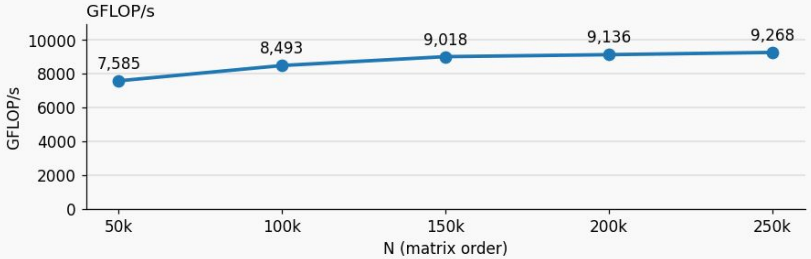
We judged each run on two metrics:

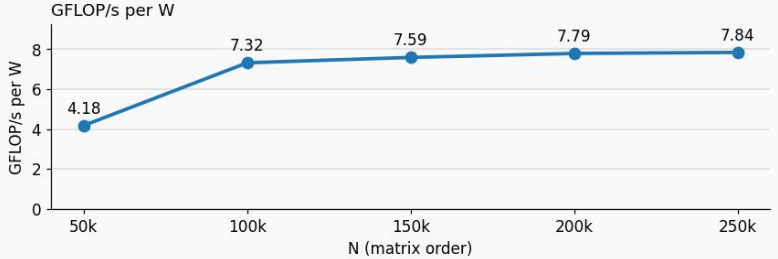
- Performance: the GFLOPS reported by HPL. The most performant configuration was the one with the highest GFLOPS.
- Energy efficiency: GFLOPS per watt. We divide the work HPL did in the timed solve ($2/3 \cdot N^3$ floating-point operations, i.e. the reported GFLOPS × solve time) by the job's total energy in joules as recorded by Slurm, which also covers setup and verification. The result is GFLOP per joule, which is the same unit as GFLOP/s per watt. The most efficient configuration was the one with the highest GFLOPS/W.

For each experiment, we selected the value of the independent variable that scored best on each metric. We then combined the best values from all experiments into two final runs: one for the highest performance and one for the highest energy efficiency.

HPL: Varying N (size of matrix)

N	NB	PxQ	Nodes	Time (s)	GFLOPS (average)	Elapsed	Joules	GFLOPS/W
50000	128	16x16	1	10.99	7.5850e+03	32	19929	4.18
100000	128	16x16	1	78.50	8.4932e+03	92	91066	7.32
150000	128	16x16	1	249.51	9.0179e+03	273	296422	7.59
200000	128	16x16	1	583.76	9.1362e+03	607	684305	7.79
250000	128	16x16	1	1123.98	9.2678e+03	1151	1329237	7.84

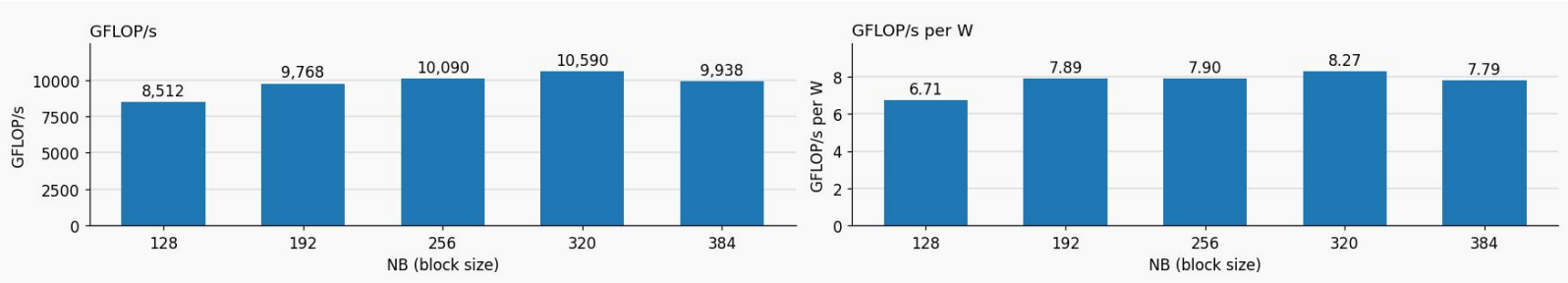




GFLOP/s per W = solve work (GFLOP) / total job energy (J); setup and verification energy count against short runs.

HPL: Varying NB (block size)

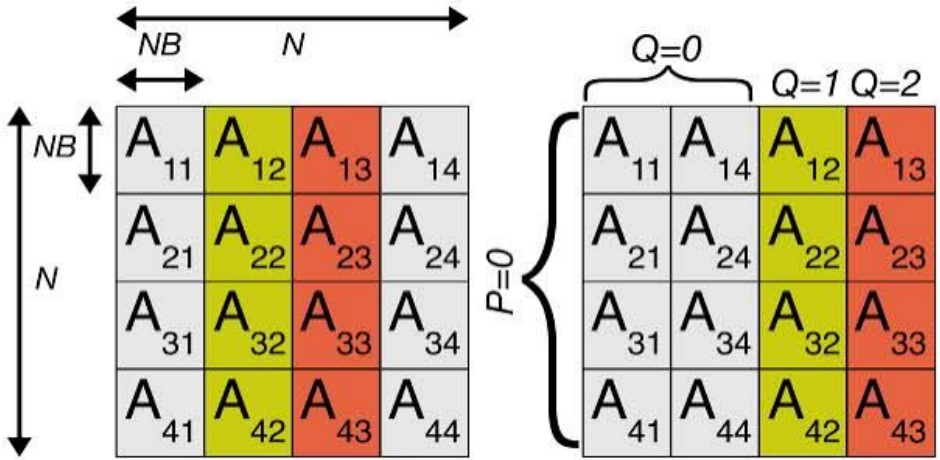
N	NB	PxQ	Nodes	Time (s)	GFLOPS (average)	Elapsed	Joules	GFLOPS/W
100000	128	16x16	1	78.32	8.512e+03	100	99395	6.71
100000	192	16x16	1	68.25	9.768e+03	89	84515	7.89
100000	256	16x16	1	66.03	1.009e+04	88	84437	7.90
100000	320	16x16	1	62.93	1.059e+04	85	80662	8.27
100000	384	16x16	1	67.08	9.938e+03	89	85564	7.79



Picking the right block size (NB)

NB is the block size HPL uses to distribute the matrix and to feed the BLAS kernels. Very small blocks give little data reuse in cache and more frequent communication; very large blocks hurt load balance.

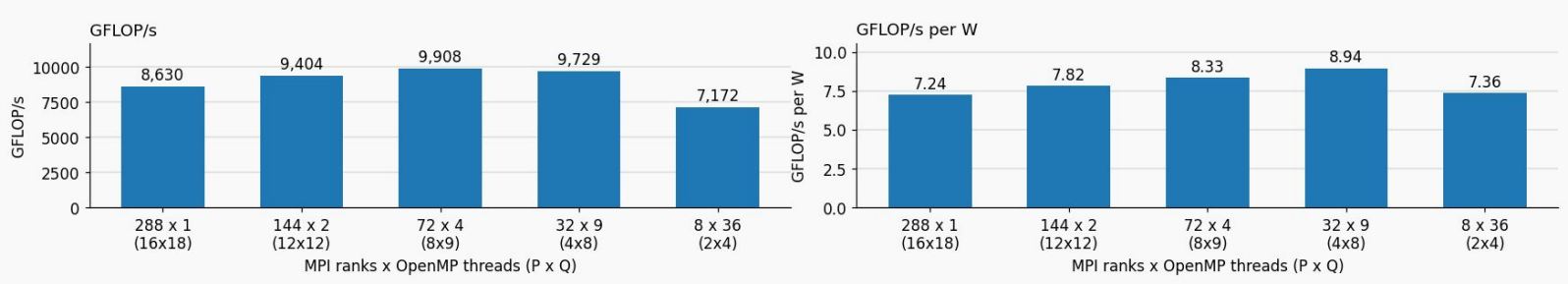
We swept NB from 128 to 384 at N = 100,000 on a 16×16 grid. The big step is 128 → 192 (+15% GFLOP/s); 320 was best in that sweep (10,590 GFLOP/s, 8.27 GFLOP/s per W). At full scale (N = 291,840, 48 × 6 layout) NB = 384 was the peak with 320 tied, so that run uses 384; the 32 × 9 efficient run uses 320.



HPL: Varying OpenMP and MPI configurations

Constants: N = 100,000 · NB = 128 · 1 node

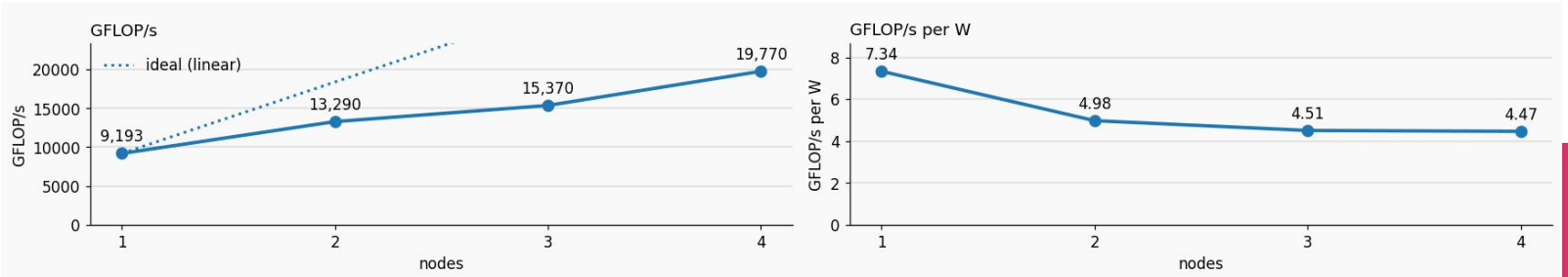
MPI ranks	OMP threads	PxQ	Time (s)	GFLOPS (average)	Elapsed	Joules	GFLOPS/W
144	2	12x12	70.89	9.404e+03	80	85291	7.82
288	1	16x18	77.25	8.630e+03	92	92032	7.24
32	9	4x8	68.52	9.729e+03	80	74605	8.94
72	4	8x9	67.29	9.908e+03	77	80075	8.33
8	36	2x4	92.95	7.172e+03	122	90636	7.36



HPL: Varying number of nodes in the job

Constants: N = 100,000 · NB = 128 · P×Q = 16×16 (256 ranks in total)

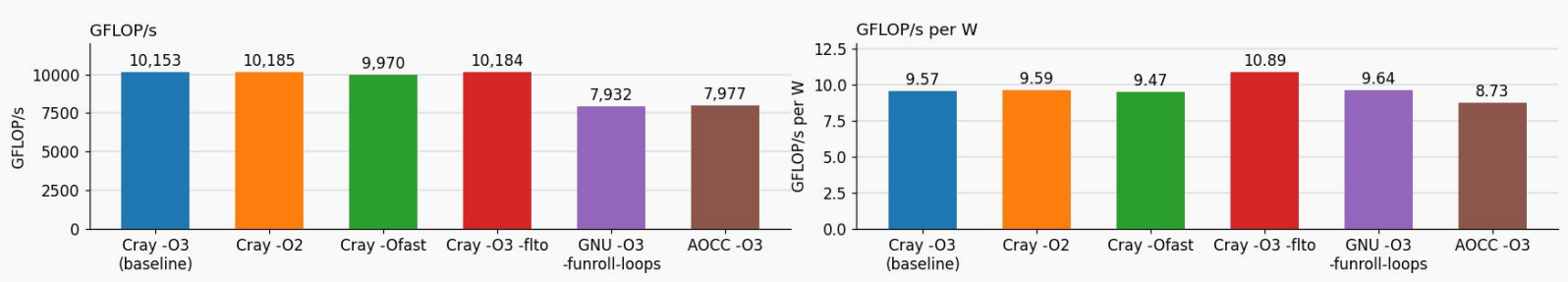
Nodes	Ranks per node	Time	GFLOPS (average)	Elapsed	Joules	GFLOPS/W
1	256	72.52	9.193e+03	93	90863	7.34
2	128	50.36	1.3239e+04	74	133831	4.98
3	86	43.35	1.537e+04	60	147840	4.51
4	64	33.72	1.977e+04	48	149308	4.47



Same N and same 256 ranks spread over more nodes: a strong-scaling test, so each node has less work and more communication.

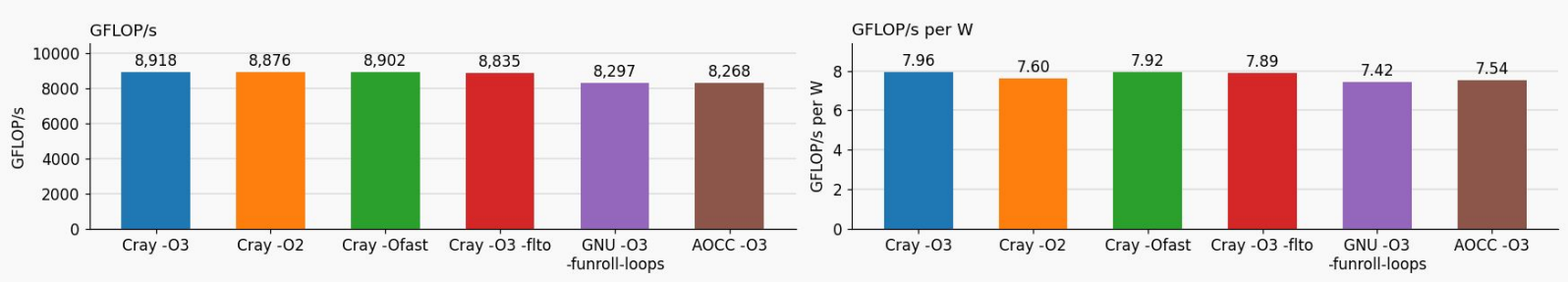
HPL: compiler flags, PxQ=2x4

Job id	Binary	N	NB	PxQ	Nodes	Time (s)	GFLOPS	Elapsed	Joules	GFLOPS/W
563364.0	Cray -O3 (baseline)	99,840	384	2×4	1	65.35	10,153	00:01:29	69,365	9.57
563364.1	Cray -O2	99,840	384	2×4	1	65.14	10,185	00:01:27	69,171	9.59
563364.2	Cray -Ofast	99,840	384	2×4	1	66.55	9,970	00:01:29	70,072	9.47
563364.3	Cray -O3 -flto	99,840	384	2×4	1	65.15	10,184	00:01:19	60,919	10.89
563364.4	GNU -O3 -funroll-loops	99,840	384	2×4	1	83.64	7,932	00:01:47	68,811	9.64
563364.5	AOCC -O3	99,840	384	2×4	1	83.18	7,977	00:01:45	76,000	8.73



HPL: compiler flags, PxQ=16x16

Job id	Binary	N	NB	PxQ	Nodes	Time (s)	GFLOPS	Elapsed	Joules	GFLOPS/W
563656.0	Cray -O3	99,840	384	16×16	1	74.40	8,918	00:01:25	83,336	7.96
563656.1	Cray -O2	99,840	384	16×16	1	74.75	8,876	00:01:25	87,336	7.60
563656.2	Cray -Ofast	99,840	384	16×16	1	74.54	8,902	00:01:25	83,798	7.92
563656.3	Cray -O3 -flto	99,840	384	16×16	1	75.09	8,835	00:01:25	84,122	7.89
563656.4	GNU -O3 -funroll-loops	99,840	384	16×16	1	79.96	8,297	00:01:29	89,393	7.42
563656.5	AOCC -O3	99,840	384	16×16	1	80.25	8,268	00:01:30	88,018	7.54



Record run

Most efficient run: GFLOPS/W = 12.79

Parameter	Value
N	250,000
NB	320
MPI ranks	32
OMP threads	9
P × Q	4 × 8, row-major mapping
Nodes	1
Compiler flags	Cray CCE 19.0.0, -O3 -flto -fopenmp (Make.cirrus_cray_omp)

Results	Value
Time	678.83 s
Elapsed	706 s
GFLOPS	1.5345e+04
Joules	814,494
GFLOPS/W	12.79

Lessons learned about performance vs energy efficiency.

Performance and efficiency are not correlated. A more performant run did not necessarily mean a more efficient run and vice versa. The clearest example is adding nodes: increasing nodes reduced time to solution but increased energy to solution.

- More nodes ≠ better efficiency. With the problem size fixed, 4 nodes ran 2.15× faster but used 64% more energy per solve. Why?
  a. Idle power adds up. Partly loaded nodes still draw substantial power for memory, networking and cooling, whether or not their cores are busy.
  b. Communication adds overhead. MPI traffic between nodes adds latency and reduces the useful computation each node does.
- Other parameters improved both. Tuning block size (NB), problem size (N) and the rank/thread layout made runs faster and more efficient, because the work finishes sooner without drawing more power.

Measurement caveat: Slurm energy includes job setup and matrix generation, while GFLOPS measures only the timed HPL solve.

High Performance Conjugate Gradient (HPCG) Experiment

Testing Configurations

Variable	Values tested
Compiler	Cray, GNU, Intel, AOCC
Local grid	104^3,128^3,144^3
MPI/OpenMP	24x12, 72x4, 288x1, plus 8x36
Nodes	1, 2, 3, 4
Tuning duration	60-120 seconds
Final duration	At least 1,800 seconds

Methodology

To avoid wasting core hours while tuning, we adopted a staged experimental approach:

• **Initial Screening:** We ran quick 60-to-120-second trials across different compilers, local grid sizes (104^3, 128^3, 144^3), and node counts (1 to 4 nodes) to spot macro trends.

• **Layout Sweeps:** We tested various MPI rank and OpenMP thread configurations (e.g., 24 × 12, 72 × 4, 288 × 1, and 8 × 36) while keeping the global grid fixed.

• **Scaling Tests:** We evaluated weak scaling (keeping work per rank constant) and strong scaling (keeping the total problem fixed across 1 to 4 nodes).

• **The Final Validation:** We locked in the best configurations and ran **30-minute runs** with GNU, including the 144 × 2 sweep winner. All passed validation, with energy from Slurm accounting.

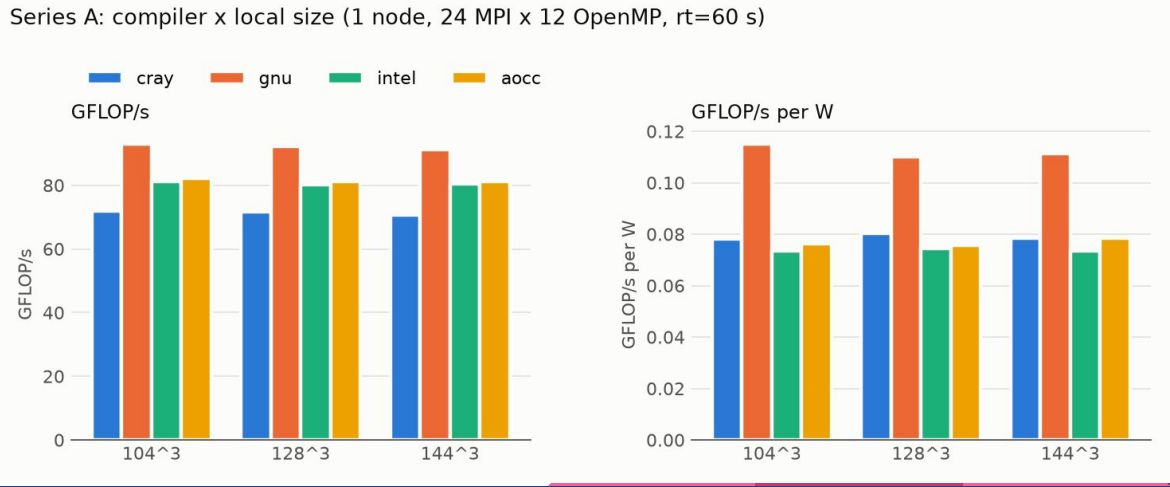
Compiler: GNU is the fastest and most efficient compiler at every problem size

(Constants: 1 node · 24 MPI × 12 OpenMP · 60 s)

Job ID	Compiler	Size	Time	Nodes	GFLOPS (average)	Elapsed	Joules	GFLOPS/W
570118	Cray	104^3	60 sec	1	71.95	00:01:21	74428	0.0783
570121	GNU	104^3	60 sec	1	**92.96**	00:01:19	63831	**0.115**
570124	Intel	104^3	60 sec	1	81.17	00:01:17	84909	0.0736
570127	AOCC	104^3	60 sec	1	82.16	00:01:23	89435	0.0762
570119	Cray	128^3	60 sec	1	71.59	00:01:39	88214	0.0803
570122	GNU	128^3	60 sec	1	92.12	00:01:37	81056	0.1102
570125	Intel	128^3	60 sec	1	80.24	00:01:40	107711	0.0745
570128	AOCC	128^3	60 sec	1	81.22	00:01:39	106451	0.0755
570120	Cray	144^3	60 sec	1	70.66	00:02:04	111474	0.0786
570123	GNU	144^3	60 sec	1	91.23	00:01:52	91863	0.1112
570126	Intel	144^3	60 sec	1	80.44	00:01:49	119083	0.0736
570129	AOCC	144^3	60 sec	1	81.27	00:01:48	111670	0.0786

We pitted Cray, GNU, Intel, and AOCC against each other under identical layouts.

- The Finding: While Cray offered slightly faster startup times, **GNU consistently delivered higher GFLOPS and superior energy efficiency (GFLOPS/W)** across almost all problem sizes.
- At a **24x12** layout, GNU outperformed Cray by roughly **29%** in throughput while maintaining an efficiency edge. Consequently, GNU became our default compiler for the final push.



MPI/OpenMP: Pure MPI (288 × 1) doubles performance over our original 24 × 12 layout

(Constants: 1 node · 384 × 384 × 512 for every layout · 90 s runs · 24×12 and 8×36 run with Cray only)

Job ID	Compiler	MPI x OMP	MPI grid	Local	GFLOPS	Elapsed	Joules	GFLOPS/W	MG (s)	SpMV (s)	DDOT (s)	Allreduce avg (s)
570136	Cray	24x12	3x2x4	128x192x128	69.9	00:02:31	136,375	0.0774	89.7	8	0.8	1.224
570134	Cray	288x1	6x6x8	64x64x64	136.22	00:02:08	133,629	0.1305	75.7	13.7	1.3	0.924
570206	GNU	288x1	6x6x8	64x64x64	135.75	00:02:09	130,930	0.1337	75.8	13.7	1.5	0.98
570135	Cray	72x4	3x3x8	128x128x64	130.15	00:02:05	135,122	0.1204	79.7	13.8	1.2	1.906
570207	GNU	72x4	3x3x8	128x128x64	129.97	00:02:06	131,654	0.1244	79.4	14.6	0.7	1.385
570137	Cray	8x36	2x2x2	192x192x256	26.9	00:03:58	189,982	0.0337	98.9	3.2	0.3	0.129

Pure MPI (288 × 1) delivered almost twice the performance of 24 × 12 and five times that of 8 × 36. GNU and Cray performed similarly with pure MPI, but GNU used slightly less power.

Why pure MPI performs better

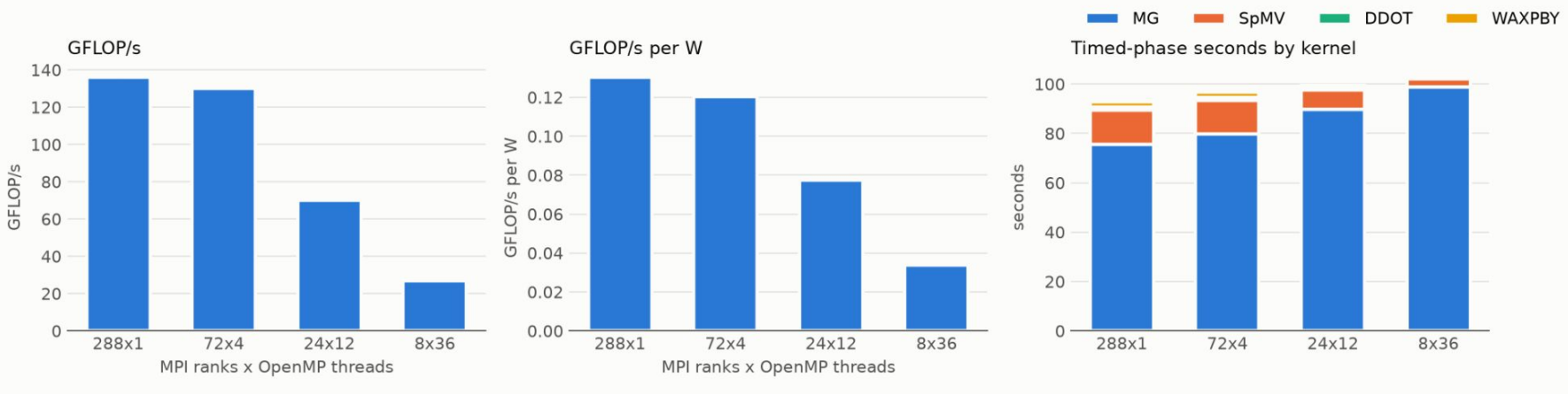
One of the most eye-opening discoveries was the dominance of pure MPI over hybrid threading:

• The Comparison: Comparing 288 × 1 (pure MPI) against hybrid setups like 24 × 12 and 8 × 36 revealed that pure MPI was up to 1.95x faster than 24 × 12 and 5x faster than 8 × 36, while using the same number of CPU cores.

• The Reason: Pure MPI performed better largely because the reference HPCG symmetric Gauss–Seidel smoother does not use OpenMP parallelism. With multiple threads per rank, much of the available threading cannot help during this phase. Using more independent MPI ranks allowed us to use the available cores more effectively, nearly doubling performance compared with our original 24 × 12 setup.

• The Refinement: However, 144 ranks with 2 cores each (one thread per rank) matched 288 × 1 at 7% lower power, which is our most efficient final result (0.1376 GFLOPS/W).

Series B: MPI x OpenMP layout, global 384x384x512 (cray, 1 node)



Weak Scaling: 4 nodes give 3.9× the performance at 97% efficiency

(Constants: Cray · 24 × 12 per node · 144³ per rank · 120 s runs)

Job ID	Compiler	Nodes	Ranks	Local	GFLOPS	GFLOPS/node	Efficiency vs 1 node	Elapsed	Joules	GFLOPS/W
570141	Cray	1	24	144x144x144	70.91	70.91	100%	00:02:59	158,245	**0.0802**
570142	Cray	2	48	144x144x144	141.26	70.63	100%	00:03:03	334,184	0.0774
570712	Cray	3	72	144x144x144	211.09	70.36	99%	00:03:04	515,198	0.0754
570143	Cray	4	96	144x144x144	**274.13**	68.53	97%	00:03:08	688,450	0.0749

Four nodes delivered nearly 3.9 times the throughput of one node while retaining 97% of the original efficiency. The trade-off brought about a modest 7% drop in performance per watt.

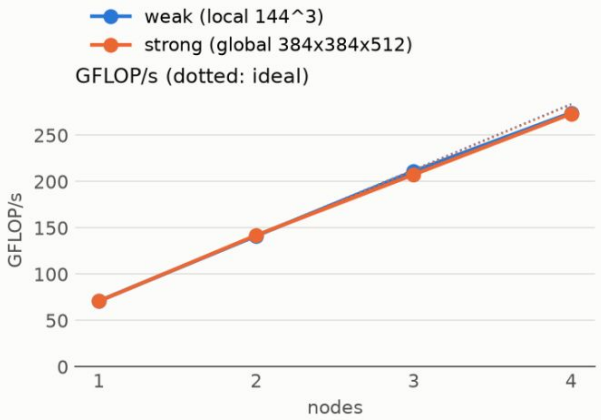
Strong Scaling: 3.9× throughput at 96% efficiency

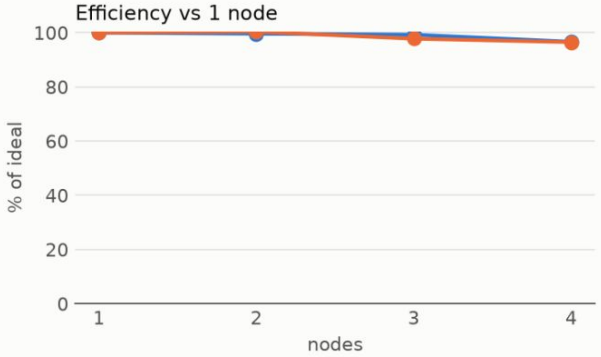
(Constants: Cray · 24 × 12 per node · global 384 × 384 × 512 · 120 s runs.)

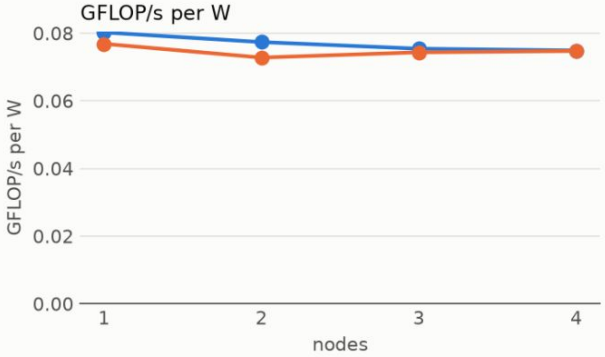
Job ID	Compiler	Nodes	Ranks	Local	GFLOPS	GFLOPS/node	Efficiency vs 1 node	Elapsed	Joules	GFLOPS/W
570147	Cray	1	24	128x192x128	70.71	70.71	100%	00:03:09	173,923	0.0768
570148	Cray	2	48	96x128x128	141.92	70.96	100%	00:02:36	304,245	0.0728
570714	Cray	3	72	128x128x64	207.40	69.12	98%	00:02:22	396,244	0.0743
570149	Cray	4	96	96x128x64	272.94	68.23	96%	00:02:18	504,109	0.0747

Even with the total problem size fixed, using four nodes delivered 3.9 times the throughput of one node, retaining 96% of its previous efficiency. This proves that Cirrus handles multi-node communication overhead exceptionally well for HPCG.

Series C: scaling over 1-4 nodes (cray, 24 MPI x 12 OpenMP per node)





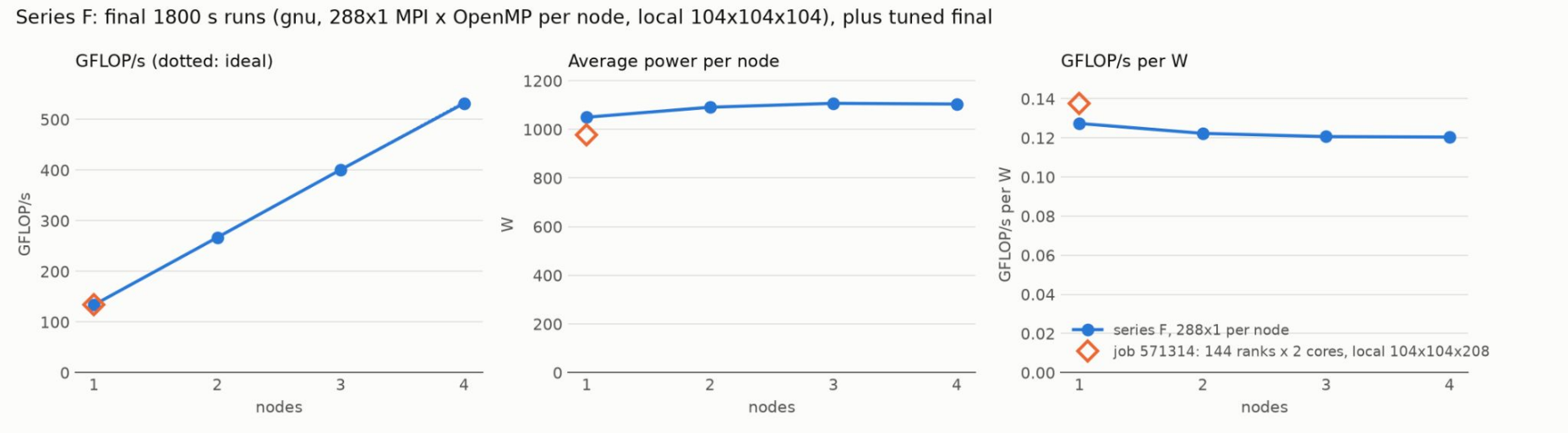


Final runs: 4 nodes give 4× the performance for 6% more energy per unit of work

(GNU · 30-minute runs · 288 MPI × 1 per node and 104³ per rank, except 571314: 144 ranks × 2 cores, 104×104×208 per rank)

Job ID	Compiler	Nodes	Local	GFLOPS	CG time (s)	Elapsed	Joules	Avg W	GFLOPS/W	HPCG data / RAM	1800 s met
570229	GNU	1	104x104x104	133.72	1805	00:32:07	2,023,146	1050	0.1274	29%	Yes
570230	GNU	2	104x104x104	266.87	1809	00:32:40	4,277,062	2182	0.1223	29%	Yes
570710	GNU	3	104x104x104	400.64	1853	00:33:17	6,632,256	3321	0.1206	29%	Yes
570231	GNU	4	104x104x104	**531.84**	1862	00:33:32	8,886,707	4417	0.1204	29%	Yes
571314	GNU	1	144×2, 104x104x208	134.55	1834	00:32:29	1,905,102	977	**0.1376**	29%	Yes

With the 288 × 1 layout, our final runs scaled nearly perfectly: using four nodes delivered almost four times the throughput at around 133 GFLOP/s per node, while one node was about 6% more energy-efficient. In later batches, we also found that running 144 ranks with 2 cores each on one node (job 571314) kept the same speed at 7% lower power, giving our best result: 0.1376 GFLOPS/W.


Series F: final 1800 s runs (gnu, 288x1 MPI x OpenMP per node, local 104x104x104), plus tuned final
GFLOP/s (dotted: ideal)
GFLOP/s
nodes
Average power per node
W
nodes
GFLOP/s per W
GFLOP/s per W
nodes
series F, 288x1 per node
job 571314: 144 ranks x 2 cores, local 104x104x208

HPCG: Record run

Most efficient: GFLOPS/W = 0.137647

Parameter	Value
Local grid	104x104x208
OMP threads per rank	1
MPI ranks	144 (2 cores each)
MPI process grid	4x6x6
Nodes	1
Compiler flags	GNU -O3 -fopenmp -funroll-loops -ftree-vectorize

Results	Value
Time	1834.13
Elapsed	1949
GFLOPS	134.547
Joules	1,905,102
GFLOPS/W	0.137647

Additional (most performant): GFLOPS = 531.837

Parameter	Value
Local grid	104x104x104
OMP threads per rank	1
MPI ranks	1152
MPI process grid	8x12x12
Nodes	4
Compiler flags	GNU -O3 -fopenmp -funroll-loops -ftree-vectorize

Results	Value
Time	1862
Elapsed	2012
GFLOPS	531.84
Joules	8886707
GFLOPS/W	0.1204

Lessons learned about performance vs energy efficiency.

Using every core did not give the best energy efficiency. Our most efficient final run kept roughly the same throughput with fewer active ranks and lower node power.

- **More active cores had diminishing returns.** At the same total problem size, 144 single-threaded MPI ranks matched 288 ranks at about 134 GFLOPS while using 7% less node power. This suggests a limit to useful parallelism within the node, although we did not isolate the bottleneck.
- **Good scaling did not guarantee better efficiency.** Four nodes handled four times the problem size at almost four times the throughput, but GFLOPS/W fell about 6% because power grew faster than throughput. Adding nodes supported larger workloads with a small efficiency cost.
- **The best configuration depends on the goal.** The 144-rank layout gave our best single-node efficiency, about 8% above 288 ranks. Four nodes gave our highest throughput. We still need to test whether the 144-rank layout retains its efficiency advantage across nodes.
- **Why does the 144x2 hybridism won at the end.** We theorize that it is the best compromise between exposing enough MPI parallelism and avoiding the overhead of too many MPI processes. This serves as a great starting point for future investigations.

Measurement caveat: Power averages use Slurm energy over matching elapsed time, including setup and validation. We did not isolate which components caused the extra multi-node power.

Differences between compute-bound (HPL) and memory-bound (HPCG) workloads

HPL reuses dense matrix blocks in cache. HPCG works with sparse matrices and depends more on memory bandwidth.

- Our one-node runs reached about 15,345 GFLOPS in HPL and 134.5 GFLOPS in HPCG at similar power. The scores reflect different algorithms and amounts of arithmetic per byte moved.
- Cray optimisation flags had little effect on our HPL tests, where most computation runs in the precompiled BLAS library. GNU gave HPCG about 29% more throughput than Cray in the 24 × 12 tests.
- Larger matrices helped HPL. Larger local grids did not consistently improve HPCG.
- For HPCG, more MPI ranks helped avoid the reference smoother's threading limit. The 144-rank final then kept similar throughput to 288 × 1 while using about 7% less power.
- In the Cray scaling tests, HPCG retained 96–97% scaling efficiency on four nodes.

GFLOPS/W helps compare configurations within each benchmark. Across HPL and HPCG, it also reflects their different workloads.

Sources

1. https://icl.utk.edu/hpl/faq/index_print.html
2. https://hpc-ga-bench.gforge.uni.lu/
3. https://www.netlib.org/benchmark/hpl/tuning.html
4. https://github.com/hpcg-benchmark/hpcg
5. https://docs.cirrus.ac.uk/user-guide/hardware/



Appendix A: Early compiler and grid tests

Job ID	Compiler	Local Grid Size	Nodes	Setup Time	Elapsed	GFLOPS	Total Bandwidth	Memory	CG Iterations	Consumed Energy	GFLOPS / Watt
562038	Cray	104x104x104	1	**0.835 sec**	**61.96 sec**	72.16	547.24 GB/s	15.98 GB	450	**73,357 J**	0.06095
570663	Cray	128x128x128	1	**1.518 sec**	65.08 sec	71.3	540.67 GB/s	29.79 GB	250	92,753 J	0.05003
564960	Cray	144x144x144	1	**2.217 sec**	74.18 sec	71.27	540.47 GB/s	42.41 GB	200	122,255 J	0.04324
562040	GNU	104x104x104	1	1.282 sec	63.29 sec	93.07	**705.81 GB/s**	15.98 GB	600	71,190 J	**0.08274**
570680	GNU	128x128x128	1	1.599 sec	**65.00 sec**	71.33	**540.95 GB/s**	36.00 GB	250	85,504 J	**0.05422**
564961	GNU	144x144x144	1	3.436 sec	**71.67 sec**	91.13	**691.11 GB/s**	42.41 GB	250	**89,540 J**	**0.07294**

GNU was faster than Cray at 104^3 and 144^3. At 128^3, their GFLOPS were almost identical. Cray's shorter setup time did not give it a higher overall score.

Appendix B: Early grid and node tests

Job ID	Local Size	Nodes	MPI Ranks	Setup Time	Elapsed	GFLOPS	Total Bandwidth	Memory	CG Iterations	Consumed Energy (J)	GFLOPS / Watt
569774	104x104x104	1	24 sec	1.276 sec	**63.29 sec**	93.08	705.92 GB/s	15.98 GB	600	66,862 J	0.08811
569855	104x104x104	2	48 sec	1.295 sec	64.16 sec	183.96	1,395.07 GB/s	31.96 GB	600	94,037 J	0.12551
569861	104x104x104	4	96 sec	**0.899 sec**	65.74 sec	**272.78**	**2,068.58 GB/s**	77.24 GB	450	116,064 J	**0.15451**
569775	128x128x128	1	24 sec	**1.594 sec**	**66.63 sec**	69.61	527.93 GB/s	36.00 GB	250	131,062 J	0.03539
569856	128x128x128	2	48 sec	1.653 sec	66.81 sec	138.98	1,053.95 GB/s	71.99 GB	250	189,112 J	0.04910
569862	128x128x128	4	96 sec	1.661 sec	68.44 sec	**271.69**	**2,060.26 GB/s**	143.99 GB	250	249,584 J	**0.07450**
569836	144x144x144	1	24 sec	**2.336 sec**	**74.32 sec**	71.1	539.16 GB/s	51.25 GB	200	316,469 J	0.01670
569857	144x144x144	2	48 sec	2.420 sec	75.68 sec	139.78	1,060.01 GB/s	102.50 GB	200	400,002 J	0.02645
569863	144x144x144	4	96 sec	2.492 sec	75.51 sec	**280.31**	**2,125.60 GB/s**	205.00 GB	200	469,822 J	**0.04505**

More nodes increased throughput. Larger local grids did not consistently help. We used these short runs for tuning, then checked energy efficiency in the later controlled tests.

Appendix C: Findings from early tests

The early tests helped us choose GNU for the next round. Larger grids were not always faster, while extra nodes increased throughput. We followed up with controlled comparisons and 30-minute final runs.